RM2UDM: PHƯƠNG PHÁP SINH TỰ ĐỘNG BẢN MẪU CHỨC NĂNG TỪ MÔ HÌNH YÊU CẦU

Lê Văn Vinh [1,2], Đặng Đức Hạnh[2]

[1] Trường Đại học Sư phạm Kỹ thuật Vinh, Việt Nam
[2] Trường Đại học Công nghệ, ĐHQGHN, Việt Nam

{21028005 | hanhdd}@vnu.edu.vn

TÓM TẮT— *Việc tạo nhanh các bản bản mẫu chức năng từ đặc tả yêu cầu chức năng phần mềm có ý nghĩa trong việc phát triển nhanh và đảm bảo chất lượng sản phẩm phần mềm. Tuy nhiên, đến nay việc tự động hóa hoạt động này vẫn đang là thách thức vì khoảng cách ngữ nghĩa giữa dạng đặc tả yêu cầu bằng ngôn ngữ tự nhiên hay ngôn ngữ bán hình thức như UML/OCL và dạng diễn đạt bằng ngôn ngữ lập trình cài đặt bản mẫu, tức mã nguồn chương trình. Trong bài báo này, chúng tôi giới thiệu một phương pháp hướng đến giải quyết vấn đề này. Trong phương pháp này, các yêu cầu chức năng đầu vào được đặc tả ở dạng mô hình yêu cầu, sử dụng các biểu đồ lớp và biểu đồ hoạt động UML/OCL. Tiếp đó, mô hình yêu cầu này được chuyển tự động về dạng mô hình miền hợp nhất (Unified Domain Model - UDM), gắn vào ngữ cảnh phương pháp thiết kế hướng miền (Domain-Driven Design - DDD). Theo tiếp cận DDD, từ mô hình UDM bằng cách xây dựng và áp dụng các bộ chuyển đổi mô hình (model transformations) cho phép sinh tự động phần mềm thực thi đóng vai trò bản mẫu chức năng. Chúng tôi đã phát triển bộ công cụ hỗ trợ, cũng như minh họa phương pháp đề xuất trong ngữ cảnh hệ thống phần mềm CourseMan.*

Từ khóa— *Bản mẫu chức năng, Mô hình yêu cầu, UML/OCL, Thiết kế hướng miền (Domain-Driven Design).*

I. GIỚI THIỆU

Trong quá trình phát triển phần mềm, việc tạo bản mẫu chức năng (functional prototyping) đóng vai trò quan trọng trong việc diễn đạt các ý tưởng, xác minh yêu cầu và cải thiện giao tiếp giữa các bên liên quan. Bản mẫu chức năng giúp kiểm tra và tinh chỉnh các tính năng của sản phẩm trước khi triển khai thực tế, từ đó giảm thiểu rủi ro, tối ưu hóa quy trình phát triển, đảm bảo rằng sản phẩm cuối cùng sẽ đáp ứng được yêu cầu của người dùng và đạt được chất lượng mong muốn [1]. Để tạo ra một bản mẫu chức năng tốt đáp ứng kỳ vọng của các bên liên quan cần có sự kết hợp giữa tự động hóa và sự can thiệp của con người, sử dụng các công cụ và phương pháp linh hoạt, cùng với quy trình phát triển và quản lý dự án hiệu quả. Việc tạo nhanh các bản bản mẫu chức năng từ đặc tả yêu cầu chức năng phần mềm có ý nghĩa trong việc phát triển nhanh và đảm bảo chất lượng sản phẩm phần mềm. Tuy nhiên, đến nay việc tự động hóa hoạt động này vẫn đang là thách thức vì khoảng cách ngữ nghĩa giữa dạng đặc tả yêu cầu bằng ngôn ngữ tự nhiên hay ngôn ngữ bán hình thức như UML/OCL [2] và dạng diễn đạt bằng ngôn ngữ lập trình cài đặt bản mẫu, tức mã nguồn chương trình.

Nhiều bài báo đã tập trung vào các phương pháp và công cụ giúp sinh tự động các bản mẫu chức năng từ đặc tả yêu cầu. Trong bài báo [3], nhóm tác giả đề xuất sử dụng IFML biểu diễn mô hình luồng tương tác, từ đó sinh tự động giao diện người dùng (GUI). Trong bài báo [4], kỹ thuật sinh tự động GUI từ mô hình ca sử dụng được đề xuất. Bằng cách sử dụng phương pháp tiếp cận theo mô hình được đề xuất trong [5], [6] sinh tự động GUI để phát triển nhanh các ứng dụng Internet các ứng dụng hỗ trợ lập trình nhanh các hệ thống điều khiển. Một số bài báo khác đề xuất kỹ thuật sinh tự động GUI từ mô hình miền (Apache Isis [7] và OpenXava [8]), sinh tự động GUI từ các biểu đồ UML bao gồm biểu đồ lớp, biểu đồ hoạt động hay biểu đồ tuần tự và các đặc tả ca sử dụng [9], [10]. Gần đây, phương pháp thiết kế phần mềm hướng miền [11], [12] đã đạt được những kết quả trong việc thu hẹp khoảng cách ngữ nghĩa giữa mô hình yêu cầu và bản cài đặt bản mẫu chức năng. Tuy nhiên, để hướng đến sinh tự động bản mẫu chức năng vẫn còn một số thách thức đặt ra cần được giải quyết: (1) Đặc tả chính xác mô hình yêu cầu hướng miền; (2) Xây dựng các bộ chuyển đổi để sinh tự động mã nguồn cài đặt bản mẫu từ mô hình yêu cầu.

Trong bài báo này, chúng tôi đề xuất kỹ thuật sinh tự động bản mẫu chức năng dựa vào bộ chuyển đổi mô hình. Cụ thể, chúng tôi phát triển ngôn ngữ chuyên biệt miền (Domain-Specific Languages - DSLs) để biểu diễn mô hình UDM tích hợp các khía cạnh cấu trúc và hành vi tương ứng với các biểu đồ lớp và biểu đồ hoạt động của mô hình yêu cầu (Requirement Model - RM) đầu vào. Tiếp đó, chúng tôi tập trung xây dựng bộ chuyển đổi từ mô hình RM sang mô hình UDM (RM2UDM) tạo điều kiện thuận lợi cho việc tự động hóa tạo bản mẫu chức năng. Chúng tôi phát triển bộ công cụ hỗ trợ phương pháp dựa trên Eclipse sử dụng các ngôn ngữ chuyển đổi mô hình ATL [13] và Acceleo [14]. Phương pháp được minh họa và đánh giá trong ngữ cảnh hệ thống quản lý khóa học CourseMan.

Phương pháp đề xuất trong bài báo này có ý nghĩa gia tăng tự động hóa trong phát triển phần mềm, đồng thời góp phần hoàn thiện phương pháp phát triển phần mềm theo DDD, giúp cho việc cập nhật, tiến hóa bản mẫu chức năng phần mềm trở nên dễ dàng hơn. Tóm lại, bài báo có bốn góp chính sau:

- Đề xuất một ngôn ngữ DSL để biểu diễn mô hình UDM;
- Xây dựng bộ chuyển đổi RM2UDM cho phép chuyển đổi mô hình yêu cầu sang mô hình hợp nhất UDM để sinh tự động bản mẫu chức năng;
- Phương pháp sinh tự động bản mẫu chức năng chức năng từ mô hình yêu cầu;
- Công cụ hỗ trợ được triển khai, chứng minh tính hiện thực hóa của phương pháp trong thực tế.

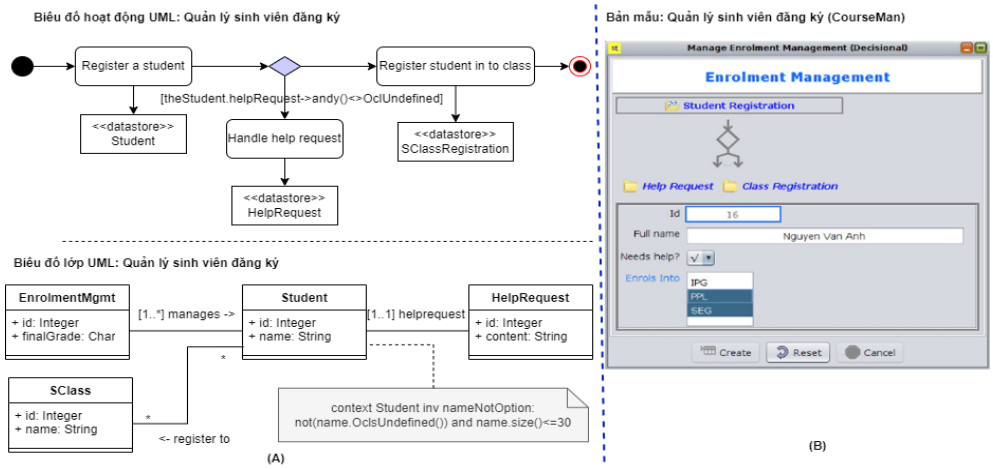
Phần còn lại của bài báo được tổ chức như sau: Phần 2 trình bài về kiến thức nền tảng và ví dụ thúc đẩy - Hệ thống quản lý đăng ký khóa học. Phần 3 trình bày tổng quan về phương pháp nghiên cứu và siêu mô hình UDML. Phần 4 trình bày phương pháp xây dựng mô hình UDM từ mô hình yêu cầu và phát triển bộ chuyển đổi RM2UDM. Phần 5 trình bày công cụ hỗ trợ và thực nghiệm sinh tự động bản mẫu chức năng. Phần 6 trình bày các công việc liên quan. Phần cuối cùng là kết luận và phác thảo hướng nghiên cứu trong tương lai.

II. KIẾN THỨC NỀN TẢNG VÀ VÍ DỤ THÚC ĐẨY

Phần này trình bày chi tiết ví dụ làm động lực thúc đẩy cho những đóng góp chính của bài báo và những kiến thức nền tảng về đặc tả mô hình miền yêu cầu, mô hình miền hợp nhất để sinh tự động bản mẫu chức năng.

A. Ví dụ thúc đẩy

Chúng tôi tập trung vào Hệ thống quản lý khóa học (`CourseMan`) làm ví dụ thúc đẩy. Hình 1 (A) biểu diễn mô hình yêu cầu đầu vào cho Hệ thống `CourseMan` ở dạng biểu đồ lớp và biểu đồ hoạt động UML/OCL: Lớp `Student` đại diện cho các sinh viên đăng ký học tại một cơ sở giáo dục; Lớp `SClass` đại diện cho loại lớp sinh viên để sinh viên lựa chọn; Lớp liên kết `SClassRegistration` nắm bắt thông tin chi tiết về liên kết nhiều-nhiều giữa `Student` và `SClass`; Lớp `HelpRequest` thu thập dữ liệu về thông tin trợ giúp được cung cấp cho sinh viên. Hình 1 (B) hiển thị giao diện bản mẫu chức năng được sinh tự động từ mô hình yêu cầu giúp giảm thời gian và công sức cho việc phát triển phần mềm, đồng thời đảm bảo tính nhất quán giữa mô hình UDM và mã được tạo ra của các bản mẫu.



Hình 1. (A) Mô hình yêu cầu cho Hệ thống CourseMan; (B) Giao diện bản mẫu chức năng được tạo tương ứng.

B. Đặc tả yêu cầu chức năng

Đặc tả yêu cầu chức năng phần mềm hướng đến xác định rõ ràng các tính năng cần phải triển khai và các yêu cầu cụ thể mà sản phẩm phải đáp ứng, bao gồm mô tả chức năng của hệ thống, giao diện người dùng, yêu cầu kỹ thuật và các ràng buộc hoạt động của phần mềm [15]. Có nhiều phương pháp đặc tả yêu cầu chức năng bao gồm nhiều kỹ thuật và công cụ khác nhau nhằm mô tả chính xác và chi tiết các yêu cầu chức năng của một hệ thống phần mềm (Ngôn ngữ tự nhiên, UML, DSL, v.v.). Nghiên cứu trong [16] trình bày các phương pháp và kỹ thuật để thiết kế đặc tả chức năng cho các hệ thống phức tạp. Các công cụ mô hình hóa UML/OCL [2] được chuẩn hóa và sử dụng rộng rãi trong phát triển phần mềm để đặc tả yêu cầu chức năng. Sử dụng ngôn ngữ chuyên biệt miền (DSL) để đặc tả chức năng phần mềm mang lại nhiều lợi ích về tính chuyên biệt, đơn giản hóa và hỗ trợ tự động hóa, đồng thời giúp cải thiện hiệu quả và chất lượng của quá trình phát triển phần mềm [17], [18], [19]. Tuy nhiên, đến nay việc sinh tự động bản mẫu chức năng từ đặc tả chức năng vẫn đang thách thức, một mặt do tính thiếu chính xác của ngôn ngữ mô hình hóa, mặt khác, do khoảng cách ngữ nghĩa của ngôn ngữ mô hình hóa và các ngôn ngữ lập trình.

Sử dụng mô hình yêu cầu trong phát triển phần mềm không chỉ đảm bảo tính toàn vẹn và hiệu quả của quá trình phát triển mà còn hỗ trợ hiệu quả các hoạt động quản lý, kiểm thử và bảo trì [20]. Trong phạm vi nghiên cứu này, chúng tôi tập trung vào mô hình yêu cầu được đặc tả ở dạng biểu đồ lớp và biểu đồ hoạt động UML/OCL.

C. Mô hình miền hợp nhất UDM

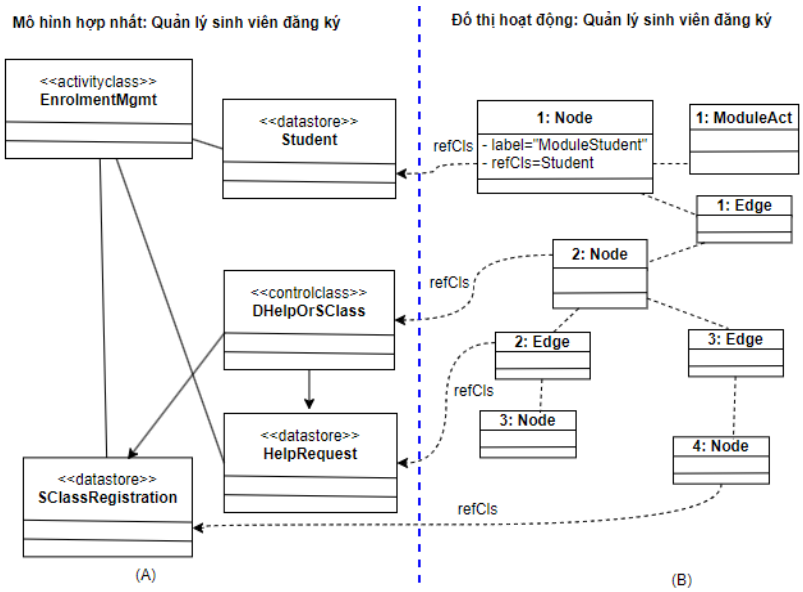
Thiết kế hướng miền (Domain-Driven Design - DDD) [11], [12] nhằm mục đích phát triển lặp lại phần mềm xung quanh mô hình miền ứng dụng. Mô hình miền ứng dụng một mặt nắm bắt triệt để các yêu cầu về miền, mặt khác đảm bảo tính khả thi về mặt kỹ thuật để triển khai. Theo phương pháp DDD, mô hình miền là thành trung tâm của phần mềm, chứa đựng hai tính năng chính như: (1) tính khả thi: Mô hình miền thực thi được như mã nguồn, và (2) tính thỏa mãn: Mô hình miền đáp ứng các yêu cầu miền được diễn đạt bằng ngôn ngữ khách hàng và chuyên gia miền.

Trong các nghiên cứu gần đây [21], [22], [23], chúng tôi đã phát triển phương pháp JDA sử dụng mô hình hóa chuyên biệt miền để thiết kế và phát triển hệ thống phần mềm. Phương pháp này áp dụng các nguyên tắc của DDD để tạo ra các

ứng dụng mà cấu trúc và hành vi phản ánh chính xác các yêu cầu nghiệp vụ. Phương pháp JDA đề xuất một số ngôn ngữ chuyên biệt miền dựa trên chú thích (aDSL) được nhúng vào ngôn ngữ lập trình hướng đối tượng, bao gồm các DSL biểu diễn các mô hình miền như sau:

- DSL biểu diễn tích hợp các khía cạnh hành vi và cấu trúc vào mô hình miền (Activity Graph Language: AGL) [21].
- DSL biểu diễn các lớp miền trong mô hình miền, hỗ trợ tự động hóa việc tạo ra không gian hành vi dựa trên không gian trạng thái của lớp miền (Domain Class Specification Language: DCSL) [22].
- DSL biểu diễn các mô-đun bản mẫu hoặc phần mềm từ các lớp miền đã được định nghĩa thông qua DCSL (Module Configuration Class Language: MCCL) [23].

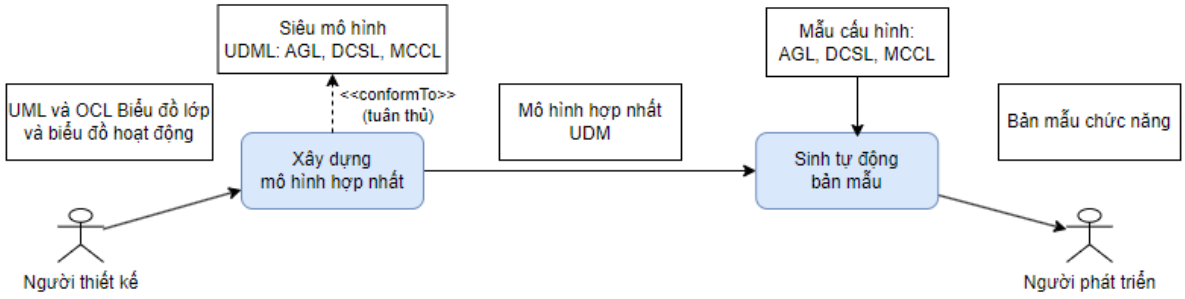
Ví dụ. Sử dụng mô hình yêu cầu ở dạng biểu đồ hoạt động và biểu đồ lớp UML của Hệ thống `CourseMan` được minh họa trong Hình 1 (A). Hình 2 (A) minh họa mô hình hợp nhất UDM chứa đựng khía cạnh hành vi tương ứng với siêu mô hình AGL. Mô hình UDM bao gồm năm lớp miền và các quan hệ kết hợp. Lớp `EnrolmentMgmt` tương ứng với biểu đồ hoạt động, trong khi lớp `DHelpOrSClass` là lớp quyết định nắm bắt logic quyết định cụ thể của miền. Ba lớp còn lại là các lớp dữ liệu đóng vai trò kho dữ liệu, tương ứng với ba lớp miền trong biểu đồ lớp UML thuộc mô hình yêu cầu. Có ba quan hệ kết hợp giúp liên kết lớp hoạt động `EnrolmentMgmt` với các lớp dữ liệu, ràng buộc các mô-đun của chúng. Hai quan hệ kết hợp còn lại liên kết lớp quyết định `DHelpOrSClass` với hai lớp dữ liệu `SClassRegistration` và `HelpRequest`, tương ứng với các kho dữ liệu kết nối với hai nốt hành động rẽ nhánh từ nốt quyết định. Hình 2 (B) minh họa đồ thị hoạt động ActivityGraph có các tham chiếu đến lớp miền thông qua liên kết được gán nhãn *refCls* tương ứng.



Hình 2. (A) Mô hình UDM của CourseMan; (B) Đồ thi hoạt động ActivityGraph.

III. TỔNG QUAN PHƯƠNG PHÁP ĐỀ XUẤT

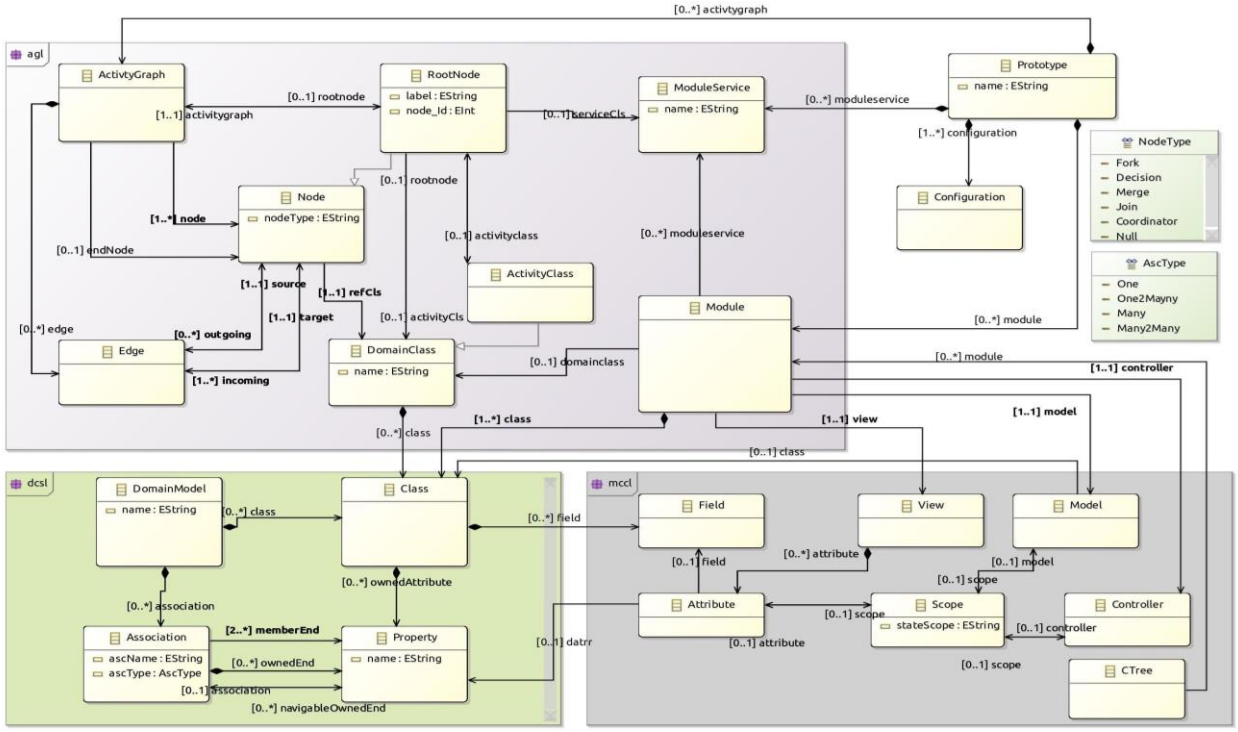
Phần này tập trung trình bày ý tưởng cơ bản của của phương pháp đề xuất: Sinh tự động bản mẫu chức năng từ mô hình yêu cầu, được biểu diễn ở dạng biểu đồ lớp và biểu đồ hoạt động UML/OCL. Hình 3 minh họa các bước chính của phương pháp: (1) Xây dựng mô hình hợp nhất và (2) Sinh tự động bản mẫu chức năng.



Hình 3. Tổng quan phương pháp đề xuất sinh bản mẫu chức năng từ mô hình yêu cầu.

Bước 1 - Xây dựng mô hình hợp nhất: Từ mô hình yêu cầu đầu vào ở dạng biểu đồ lớp và biểu đồ hoạt động UML/OCL, bước này tập trung xây dựng mô hình hợp nhất UDM. Mô hình hợp nhất này cần tuân thủ với siêu mô hình UDML được biểu diễn trong Hình 4 diễn đạt siêu mô hình UDML bao gồm các siêu khái niệm của AGL, DCSL và MCCL. Trong sự hợp nhất này, các siêu khái niệm *Software* và *Configuration* được bổ sung để diễn đạt kiến trúc tổng thể của bản mẫu.

Bước 2 - Sinh tự động bản mẫu chức năng: Mô hình UDM kết hợp với mẫu cấu hình của AGL, DCSL và MCCL tiến đến sinh tự động bản mẫu chức năng; Bản mẫu chức năng được tạo ra gửi đến chuyên gia miền, nếu có phản hồi thì mô hình UDM được cập nhật lại cấu trúc hoặc hành vi theo chuyên gia và tiếp tục vòng lặp tiếp theo nếu có. Mặt khác, nếu chuyên gia miền hài lòng, đáp ứng yêu cầu người dùng thì chu trình sẽ kết thúc.



Hình 4. Siêu mô hình UDML biểu diễn dưới dạng biểu đồ lớp.

IV. CHUYỂN ĐỔI MÔ HÌNH MIỀN YÊU CẦU SANG MÔ HÌNH HỢP NHẤT

Phần này tập trung trình bày kỹ thuật xây dựng mô hình hợp nhất (UDM) từ mô hình yêu cầu (RM) và phát triển bộ chuyển đổi mô hình RM2UDM với các luật chuyển đổi và thuật toán cài đặt.

A. Xây dựng mô hình UDM từ mô hình yêu cầu

Mô hình hợp nhất UDM bao gồm hai thành phần: (1) Biểu diễn khía cạnh cấu trúc, tương ứng với DCSL và MCCL; (2) Biểu diễn khía cạnh hành vi tương ứng với AGL.

Biểu diễn khía cạnh cấu trúc: DCSL, MCCL và lớp hoạt động. Một mô hình UDM biểu diễn khía cạnh cấu trúc là mô hình DCSL, MCCL và lớp hoạt động như sau:

- Một lớp miền c_a (gọi là lớp miền hoạt động) được xác định tương ứng với biểu đồ hoạt động đầu vào.
- Các lớp $c_1,...,c_n$ được xác định tương ứng với các lớp miền thuộc biểu đồ lớp đầu vào.
- Với $c_{i1},...,c_{ik} \in \{c_1,...,c_n\}$ thực hiện các lớp thành phần không phải nốt quyết định và không phải nốt kết hợp, thì $c_a,c_{i1},...,c_{ik}$ chứa các trường liên kết thực hiện các mối quan hệ kết thúc tương ứng của các mối quan hệ cụ thể của hoạt động liên quan.

Biểu diễn khía cạnh hành vi: Đồ thị hoạt động AGL. Biểu đồ hoạt động đầu vào biểu diễn hành vi sẽ được ánh xạ thành một lớp hoạt động (ActivityClass), được tích hợp vào mô hình miền DSL.

- Lớp hoạt động: một lớp miền đại diện cho hoạt động.
- Lớp thành phần dữ liệu (hay gọi tắt là lớp dữ liệu): một lớp miền đại diện cho mỗi kho dữ liệu.
- Lớp thành phần điều khiển (hoặc lớp điều khiển): nắm bắt trạng thái miền cụ thể của nốt điều khiển. Một lớp điều khiển đại diện (không đại diện) một nốt điều khiển được đặt tên theo (phủ định) ví dụ, lớp quyết định (không quyết định), lớp tham gia (không tham gia), v.v.
- Lớp thành phần điều phối: một lớp đại diện một nhóm nhiệm vụ và không chuyển dữ liệu của nhóm đó ra các cạnh đi ra, để thực hiện logic của nhóm.
- Liên kết dành riêng cho hoạt động: liên kết giữa mỗi cặp lớp sau: Lớp hoạt động và lớp hợp nhất (merge); Lớp hoạt động và lớp rẽ nhánh (fork); Một lớp hợp nhất (rẽ nhánh) và một lớp dữ liệu đại diện cho kho lưu trữ dữ liệu của một nốt hành động được kết nối với nốt hợp nhất (rẽ nhánh); Lớp hoạt động và một lớp dữ liệu không đại diện cho kho dữ liệu của nốt hành động được kết nối với nốt hợp nhất hoặc nốt rẽ nhánh.

B. Xây dựng bộ chuyển đổi mô hình RM2UDM

Dưới đây là nội dung thuật toán RM2UDM, được cài đặt bằng ATL [13], để chuyển đổi mô hình yêu cầu RM sang mô hình UDM. Mô hình kết quả UDM tuân thủ siêu mô UDML, do đó, được ánh xạ tự động sang mã nguồn Java để cài đặt bản mẫu chức năng.

Luật 1: Mỗi bản mẫu được xác định một lớp miền đại diện cho hoạt động trong biểu đồ lớp ánh xạ sang tên lớp hoạt động (ActivityClass) để biểu diễn hành vi (ActivityGraph) cho bản mẫu.

Luật 2: Mỗi hoạt động (Activity) trong biểu đồ hoạt động được ánh xạ sang nốt (Node), liên kết các nốt thông qua cạnh (Edge) với thể hiện lớp miền (DomainClass) và được gán nhãn tương ứng được biểu thị bằng từ khóa `refCls` và `outCls` trong UDM.

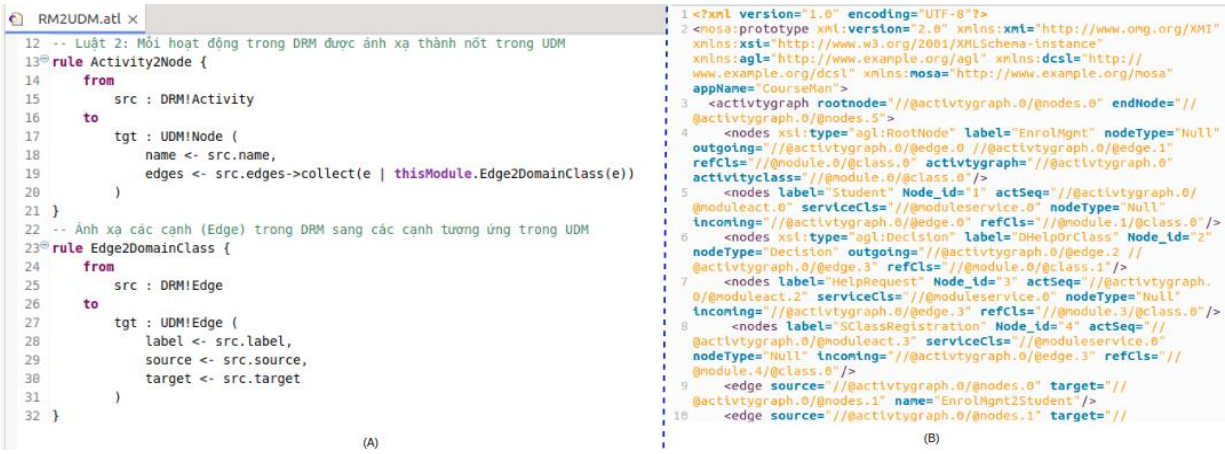
Luật 3: Mỗi thể hiện của nốt điều khiển (ControllNode) trong RM được gọi một mô hình mẫu tương ứng (Decision, Fork, Merge, Join, Sequential, Coordinate) ánh xạ sang (DomainPattern) với thể hiện cùng tên trong UDM.

Luật 4: Mỗi hành vi được tham chiếu đến DomainClass bởi Node qua cấu trúc của mô hình miền bằng cách sử dụng từ khóa `actSeq`. Điều này đảm bảo rằng trạng thái thực thi của hoạt động được đồng bộ hóa với trạng thái hiện tại của mô hình hợp nhất.

Luật 5: Mỗi thuộc tính trong lớp của RM cùng với các ràng buộc ánh xạ sang lớp và thuộc tính tương ứng, kết hợp với thể hiện DCSL của mô hình UDM.

Luật 6: Mỗi lớp của RM ánh xạ sang mô-đun tương ứng tạo thành cấu hình của lớp mô-đun sở hữu một lớp miền trong UDM.

Các luật chuyên này sẽ được diễn đạt bằng ngôn ngữ chuyển đổi mô hình sang mô hình (M2M), ở đây chúng tôi sử dụng ATL [13] để xây dựng bộ chuyển đổi mô hình RM2UDM. Hình 5 (A) mô tả một phần của luật chuyển được viết bằng ATL và Hình 5 (B) biểu diễn kết quả ánh xạ mô hình yêu cầu Hệ thống CourseMan sang mô hình UDM.



Hình 5. Các luật chuyển đổi của ATL và kết quả mô hình UDM.

Thuật toán 1. Chuyển mô hình yêu cầu sang mô hình hợp nhất bằng bộ chuyển RM2UDM

Input: Mô hình yêu cầu RM = <CD, AD> trong đó, CD: biểu đồ lớp được gắn các ràng buộc OCL; AD: biểu đồ hoạt động

Output: Mô hình hợp nhất UDM

Begin

1. Tạo lớp hoạt động với tên là nhãn ánh xạ từ lớp miền đại diện cho hoạt động trong biểu đồ lớp với đặc tả là <ActivityGraph> **(Luật 1)**

2. foreach act: Activity | act ∈ AD:

   if(act**:** là nốt điều khiển ControlNode):

   Gọi một mô hình mẫu tương ứng (Decision, Fork, Merge, Join, Sequential, Coordinate) ánh xạ thành (DomainPattern) **(Luật 3)**

   else (Định nghĩa Node thể hiện DomainClass (**Luật 2**))

   Ánh xạ nhãn Activity sang nhãn của Node và gán cho thuộc tính refCls

Ánh xạ nhãn của lớp dữ liệu tương ứng với kho dữ liệu và chúng cũng tương ứng với lớp miền trong sơ đồ lớp UML sang thuộc tính refCls của Node

Với mỗi liên kết các Activity thông qua Edge để xác định các nhãn của nó và ánh xạ sang tập thuộc tính OutClass của Node

Với mỗi hành vi được tham chiếu đến DomainClass đặc tả trong các ràng buộc OCL sang tập các hành động mô-đun gọi là thuộc tính MAct (**Luật 4**)

3. foreach cl:Class | cl ∈ CD:

3.1 Ánh xạ thuộc tính trong Class và ràng buộc sang thuộc tính tương ứng và kết hợp với thể hiện DCSL **(Luật 5)**

3.2 Ánh xạ Class của RM sang mô-đun tương ứng tạo thành cấu hình của lớp mô-đun sở hữu một lớp miền trong UDM (**Luật 6**)

End

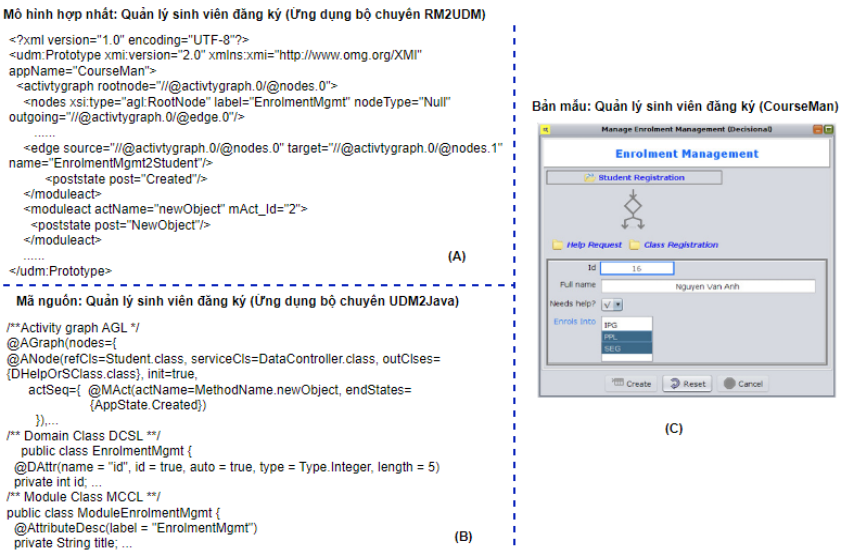
Thuật toán 1 đặc tả trình tự áp dụng các luật ánh xạ để thực thi bộ chuyển đổi RM2UDM. Thuật toán này, cũng như các thuật toán cài đặt bộ chuyển ATL khác có độ phức tạp hàm mũ, như được thảo luận và đánh giá trong [24]. Trong phạm vi bài báo này, chúng tôi quan tâm về tính đúng đắn của thuật toán và tiến hành kiểm chứng việc này thông qua kiểm thử.

Đối với bộ chuyển đổi RD2UDM của phương pháp chúng tôi đề xuất, chúng tôi giải thích phép biến đổi chính xác từ RD sang UDM theo một tập hợp các quy tắc ánh xạ từ Luật 1 đến Luật 6. Ánh xạ RD sang UDM là song ánh, tức là đối với mọi thể hiện của RM, tồn tại chính xác một thể hiện của UDM mà nó được ánh xạ đến. Vì mỗi quy tắc ánh xạ trình bày ở trên được áp dụng độc lập, là hiển nhiên vì nó trực tiếp suy ra rằng UDM không chứa bất kỳ tính năng mới nào và từ hai thuộc tính đã được ánh xạ.

V. CÔNG CỤ HỖ TRỢ VÀ THỰC NGHIỆM

Trong phần này trình bày công cụ hỗ trợ được thực thi trong Khung phần mềm JDA, đánh giá kết quả thực nghiệm phương pháp đề xuất trên ví dụ CourseMan và OrderMan. Cuối phần này là một số thảo luận về những yếu tố ảnh hưởng đến tính đúng đắn của phương pháp đề xuất.

Chúng tôi phát triển bộ công cụ hỗ trợ phương pháp đề xuất để sinh tự bản mẫu chức năng từ mô hình yêu cầu. Người thiết kế sẽ sử dụng các công cụ tạo biểu đồ lớp và biểu đồ hoạt động UML/OCL dưới dạng trực quan. Sử dụng bộ chuyển đổi thực hiện bằng ATL trên Eclipse để chuyển đổi RM2UDM. Mô hình UDM được ánh xạ sang mã nguồn Java để sinh tự động bản mẫu chức năng theo thiết kế hướng miền và thực thi tạo giao diện bản mẫu với Khung phần mềm JDA. Ví dụ minh họa áp dụng cho CouseMan, Hình 6 (A) hiển thị một phần UDM dạng XMI là kết quả của bộ chuyển đổi RD2UDM, Hình 6 (B) hiển thị một phần mã nguồn Java là kết quả của ánh xạ mô hình UDM bằng Acceleo trên Eclipse và Hình 6 (C) biểu diễn kết quả GUI bản mẫu chức năng cho CourseMan được tạo ra.



Hình 6. Sinh tự động bản mẫu chức năng từ mô hình miền yêu cầu sử dụng bộ chuyển đổi RM2UDM.

Chúng tôi thực hiện đánh giá về hiệu suất của công cụ cho Quản lý đăng ký khóa học CourseMan và Quản lý đơn hàng OrderMan với kế quả:

Đối với sử dụng bộ chuyển đổi RD2UDM thì khả năng diễn đạt của mô hình hợp nhất đáp ứng trên 95%. Đối với ánh xạ tự động từ mô hình UDM sang mã nguồn Java để cài đặt bản mẫu chức năng thì tất cả lớp miền của mô hình yêu cầu được chuyển thành các lớp trong mã nguồn và tương ứng với số mô-đun được tạo ra. Đạt chính xác mã nguồn được tạo ra trên 87% và chỉnh sửa, bổ sung thực hiện thủ công dưới 13%. Nguyên nhân, do một số mối đe dọa ảnh hưởng đến phương pháp đề xuất của chúng tôi được trình bày trong phần thảo luận tiếp theo.

Thảo luận. Chúng tôi đã áp dụng thành công phương pháp đề xuất cho nghiên cứu CourseMan và OrderMan. Điều này cho thấy khả năng ứng dụng tiềm năng của phương pháp của chúng tôi trong thực tế. Tuy nhiên, chúng tôi thảo luận về một số mối đe dọa đối với tính hợp lệ của phương pháp của chúng tôi với các khía cạnh như sau. Một là, phương pháp của chúng tôi tiếp nhận đặc tả yêu cầu đầu vào ở dạng biểu đồ UML/OCL, do đó, tính đúng đắn của phương pháp phụ thuộc vào năng lực diễn đạt của ngôn ngữ mô hình hóa này; Hai là, phương pháp đề xuất chưa đảm bảo mọi mô hình đầu vào UML/OCL đều tương ứng với một mô hình hợp nhất UDM. Hơn nữa, các ràng buộc OCL trên mô hình đầu vào cần được tương ứng một cách thủ công diễn đạt tương ứng bằng DCSL cho mô hình UDM. Những vấn đề này liên quan đến phạm vi áp dụng và tính đúng đắn của bộ chuyển đổi RM2UDM. Hiện tại tính đúng đắn của phương pháp ở khía cạnh này được đảm bảo bằng cách kiểm tra, rà soát các luật chuyển, đồng thời thực thi kiểm thử bộ chuyển đổi; Ba là, kết quả bản mẫu thực thi trong Java có thể không chính xác so với đặc tả yêu cầu đầu vào nếu có sai sót trong tạo mã nguồn Java bằng những ánh xạ tự động từ mô hình UDM. Cũng như với bộ chuyển đổi RM2UDM, chúng tôi cố gắng đảm bảo độ chính xác bằng cách rà soát, kiểm tra các luật chuyển đổi và tiến hành kiểm thử cho bộ chuyển đổi này.

VI. CÔNG VIỆC LIÊN QUAN

Thiết kế hướng miền [12], [18], [25] nhằm mục đích phát triển lặp lại phần mềm xung quanh một mô hình thực tế của miền ứng dụng để nắm bắt triệt để các yêu cầu về miền. Trong mô hình miền tiếp cận theo phương pháp DDD có xu hướng trở thành trung tâm của phần mềm nhằm mục đích tăng tính khả thi và tính thỏa mãn (mô hình miền sẽ đáp ứng các yêu cầu miền được thể hiện bằng ngôn ngữ phổ biến). Phương pháp DDD mang lại nhiều lợi ích khi phát triển các hệ thống phần mềm phức tạp. Tuy nhiên, khi yêu cầu thay đổi thì mô hình miền cũng phải thay đổi theo, đòi hỏi sự linh hoạt và khả năng cập nhật liên tục từ phía người phát triển, người thiết kế sẽ gây khó khăn cho nhà thiết kế và nhà phát triển. Trong nghiên cứu [18] về DSL, một mô hình được coi là tương đương với một đặc tả trong ngôn ngữ dành riêng cho miền văn bản và đã chỉ ra việc sinh mã nguồn trải qua nhiều bước, có nhiều chuyển đổi mô hình sang mô hình và một bước tạo mã nguồn cuối cùng, mã được tạo sau đó được biên dịch và thực thi. Các nghiên cứu trong [7], [8] đã sử dụng phương pháp DDD đề xuất sử dụng DSL cung cấp các khung làm việc giúp các nhà phát triển dễ dàng mô hình hóa nghiệp vụ phức tạp. Nhưng cũng có những khó khăn nhất định liên quan đến phạm vi ứng dụng, độ phức tạp do các hạn chế về hiệu suất và khả năng tùy chỉnh. Trong nghiên cứu gần đây, chúng tôi đã đề xuất AGL [21], DCSL [22] và MCCL [23] để thiết kế và phát triển hệ thống phần mềm bằng cách sử dụng ngôn ngữ chuyên biệt miền dựa trên chú thích (aDSL) viết bên trong ngôn ngữ lập trình hướng đối tượng. Tuy nhiên, còn tồn tại một số hạn chế như là aDSL làm cho mã khó đọc, khó hiểu khi liên lạc giữa các bên liên quan và thiếu tính linh hoạt.

Trong [4], [5], [6] các tác giả đã đề xuất việc tự động hóa thiết kế giao diện người dùng, cải thiện sự nhất quán và hiệu quả trong quy trình phát triển phần mềm đơn giản. Nhưng đối với phần mềm phức tạp còn tồn tại hạn chế như thiếu linh hoạt và khó tùy chỉnh theo các yêu cầu đặc biệt của người dùng. Sinh tự động GUI, bản mẫu từ các biểu đồ UML và các đặc tả ca sử dụng đã được đề xuất trong [3], [9], [10] với đóng góp chính là tự động chuyển đổi, phát triển công cụ hỗ trợ chuyển đổi, đảm bảo tính nhất quán và đúng đắn của mã nguồn. Tuy nhiên, chưa ai đề xuất được một phương pháp hoàn chỉnh thu hẹp khoảng cách ngữ nghĩa giữa dạng đặc tả yêu cầu bằng ngôn ngữ tự nhiên hay các ngôn ngữ bán hình thức như UML/OCL [2], [26] và các mã nguồn chương trình, tức dạng diễn đạt sử các ngôn ngữ lập trình cài đặt bản mẫu. Trong bài báo này, chúng tôi đề xuất sử dụng DSL hợp nhất các biểu đồ UML/OCL bao gồm biểu đồ lớp (cấu trúc) và biểu đồ hoạt động (hành vi). Việc kết hợp khía cạnh cấu trúc và hành vi trong đặc tả yêu cầu miền giúp tạo ra một cái nhìn toàn diện, nhất quán và chi tiết về hệ thống phần mềm, từ đó hỗ trợ quá trình phát triển, bảo trì, kiểm thử và giao tiếp hiệu quả hơn bằng bộ chuyển đổi RM2UDM để chuyển đổi mô hình yêu cầu sang mô hình thiết kế tạo điều kiện thuận lợi cho việc tự động hóa tạo bản mẫu chức năng.

VII. KẾT LUẬN

Trong bài báo này, chúng tôi giới thiệu phương pháp tạo nhanh các bản bản mẫu chức năng từ mô hình yêu cầu sử dụng các biểu đồ lớp và biểu đồ hoạt động UML/OCL. Đầu tiên, phương pháp chỉ rõ cách thức xây dựng mô hình hợp nhất UDM từ mô hình yêu cầu đầu vào RM. Tiếp đó, bộ chuyển đổi mô hình RM2UDM được phát triển với các luật chuyển đổi và thuật toán cài đặt. Chúng tôi đã minh họa, đánh giá phương pháp đề xuất trên ca nghiên cứu với Hệ thống CourseMan để thực hiện phương pháp của mình.

Như một phần của công việc tương lai, chúng tôi sẽ tập trung vào việc xem xét dạng đặc tả yêu cầu cho các tình huống thực hành khác nhau, bao gồm: các dạng biểu đồ khác, ngôn ngữ tự nhiên, v.v. Ngoài ra, tinh chỉnh làm tăng khả năng diễn đạt UDM và hoàn thiện các bộ chuyển đổi tương ứng, cũng như xem xét vấn đề tiến hóa bản mẫu là một phần của công việc tương lai của chúng tôi.

VIII. TÀI LIỆU THAM KHẢO

[1] W. Suryn, *Software Quality Engineering: A Practitioner's Approach*. John Wiley & Sons, 2013.

[2] OMG, "Unified Modeling Language version 2.5.1," OMG, formal/2015-03-01, 2017.

[3] A. M. Rosado da Cruz and J. Faria, "Automatic Generation of User Interface Models and Prototypes from Domain and Use Case Models," User Interfaces, Rita Matrai (Ed.)., Intech, 2010. doi: 10.5772/9498.

[4] I. Antović, S. Vlajić, M. Milić, D. Savić, and V. Stanojević, "Model and software tool for automatic generation of user interface based on use case and data model," *Institution of Engineering and Technology*, vol. 6, no. 6, pp. 559–

573, 2012, doi: 10.1049/iet-sen.2011.0060.

[5] M. L. Bernardi, G. A. Di Lucca, and D. Distante, "Model-driven fast prototyping of RIAs: From conceptual models to running applications," in *Proc. 2014 Int. Conf. Advances in Computing, Communications and Informatics (ICACCI)*, 2014, pp. 250–258. doi: 10.1109/ICACCI.2014.6968522.

[6] Willow, "Model-Driven Rapid Prototyping for Control Algorithms with the GIPS Framework," in *Proc. 14th. Int. Workshop Graph Computation Models (GCM)*, STAFF/GCM Session 1, 2023.

[7] Dan Haywood, "Apache Isis - Developing Domain-Driven Java Apps," *Methods & Tools: Practical knowledge source for software development professionals*, vol. 21, no. 2, pp. 40–59, 2013.

[8] Paniza, Javier, "OpenXava." [Online]. Available: https://www.openxava.org

[9] M. I. Mukhtar and B. S. Galadanci, "Automatic code generation from UML diagrams: the state-of-the-art," *Science World Journal*, vol. 13, no. 4, Art. no. 4, 2019.

[10] P. Kluisritrakul and Y. Limpiyakorn, "Generation of Java Code from UML Sequence and Class Diagrams," in *Proc. 7th Int. Conf. Information Science and Applications (ICISA 2016)*, Springer. K.J. Kim and N. Joukov (eds.), 2016, pp. 1117–1125. doi: DOI: 10.1007/978-981-10-0557-2_106.

[11] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2004.

[12] Vaughn Vernon, *Implementing Domain-Driven Design*, 1st ed. Upper Saddle River, NJ: Addison-Wesley Professional, 2013.

[13] Frédéric Jouault a, Freddy Allilaire a, Jean Bézivin a, and Ivan Kurtev, "ATL: A model transformation tool," *Science of computer programming*, vol. 72, no. 1–2, pp. 31–39, 2008, doi: 10.1016/j.scico.2007.08.002.

[14] E. Syriani, L. Luhunu, and H. Sahraoui, "Systematic mapping study of template-based code generation," *Computer Languages, Systems & Structures*, vol. 52, pp. 43–62, Jun. 2018, doi: 10.1016/j.cl.2017.11.003.

[15] A. Fatwanto and C. Boughton, "Analysis, Specification and Modeling of Functional Requirements for Translative Model-Driven Development," in *Proc. 2008 Int. Symposium Conf. Knowledge Acquisition and Modeling*, IEEE, Oct. 2008, pp. 859–863. doi: 10.1109/KAM.2008.185.

[16] O. Goubali *et al.*, "Designing Functional Specifications for Complex Systems," in *Human-Computer Interaction. Theory, Design, Development and Practice. HCI 2016. Lecture Notes in Computer Science()*, Springer, Cham, 2016, pp. 166–177. doi: 10.1007/978-3-319-39510-4_16.

[17] M. Brambilla, J. Cabot, and M. Wimmer, *Model-Driven Software Engineering in Practice*. Springer Nature, 2022.

[18] M. Fowler, *Domain-Specific Languages*. Pearson Education, 2010.

[19] M. Jeusfeld, M. Jarke, and J. Mylopoulos, *Metamodeling for Method Engineering*. MIT Press, 2009.

[20] K. Wiegers and C. Hokanson, *Software Requirements Essentials: Core Practices for Successful Business Analysis*. Addison-Wesley Professional, 2023.

[21] D.-H. Dang, D. M. Le, V.-V. Le, "Agl: Incorporating behavioral aspects into domain-driven design," *Information and Software Technology*, vol. 163, 2023, doi: 10.1016/j.infsof.2023.107284.

[22] LE, Duc Minh; DANG, Duc-Hanh; NGUYEN, Viet-Ha, "On domain driven design using annotation-based domain specific language," *Computer Languages, Systems & Structures*, vol. 54, pp. 199–235, 2018, doi: 10.1016/j.cl.2018.05.001.

[23] D.M. Le, D.-H. Dang, V.-H. Nguyen, "Generative software module development for domain-driven design with annotation-based domain specific language," *Information and Software Technology*, vol. 120, pp. 106–239, 2022, doi: 10.1016/j.infsof.2019.106239.

[24] S. Götz and M. Tichy, "Investigating the Origins of Complexity and Expressiveness in ATL Transformations.," *Journal of Object Technology*, vol. 19, no. 2, p. 12:1-21, 2020, doi: 10.5381/jot.2020.19.2.a12.

[25] S. K. Jaiswal and R. Agrawal, "Domain-Driven Design (DDD)- Bridging the Gap between Business Requirements and Object-Oriented Modeling," *Int. Innovative Research in Engineering and Management*, vol. 11, no. 2, Art. no. 2, 2024.

[26] E. Foster and B. T. Jr, *Software Engineering: A Methodical Approach, 2nd Edition*, 2nd ed. New York: Auerbach Publications, 2021. doi: 10.1201/9780367746025.

RM2UDM: A METHOD FOR AUTOMATICALLY GENERATING FUNCTIONAL PROTOTYPES FROM REQUIREMENT MODELS

Le Van Vinh, Dang Duc Hanh

ABSTRACT— *The rapid creation of functional prototypes from software functional requirements specifications is significant for fast development and ensuring software product quality. However, automating this activity remains a challenge due to the semantic gap between requirement specifications in natural language or semi-formal languages like UML/OCL and the program source code, the way of expression using programming languages to implement prototypes. In this paper, we introduce a method aimed at addressing this issue. In this method, the input functional requirements are specified in the form of requirement models, using UML/OCL class and activity diagrams. These requirement models are then automatically transformed into a Unified Domain Model (UDM) within the context of Domain-Driven Design (DDD). Following the DDD approach, from the UDM, by constructing and applying model transformations, it is possible to automatically generate executable software that serves as functional prototypes. We have developed a supporting tool and illustrated the proposed method in the context of the CourseMan software system.*